

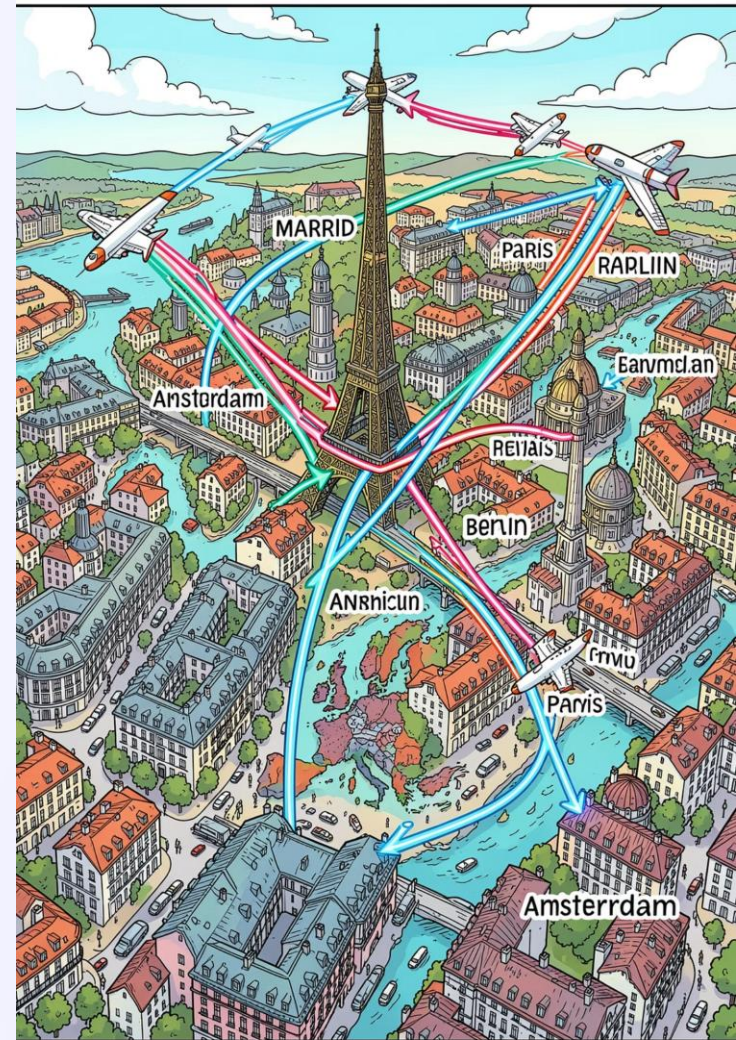
PROYECTO INTERMODULAR · 2º DAM

UrTriply

Planificador de viajes con
comunidad integrada

Paula Izurrategui López

Tutora: Macarena Cuenca Carbajo



Índice

01

Fundamentación

03

Objetivos

05

Temporalización

07

Conclusiones

02

Destinatarios

04

Metodología

06

Recursos

08

Bibliografía y Webgrafía

1. FUNDAMENTACIÓN

UrTriply: Descripción del Proyecto

App Android nativa (Kotlin + Jetpack Compose) que genera propuestas de viaje a capitales europeas desde MAD según presupuesto, viajeros, fechas y preferencias, con itinerario detallado y comunidad social integrada.

Problemas Identificados

- Presupuestación fragmentada en múltiples plataformas
- Sin flexibilidad temporal ni validación rápida
- Aislamiento de experiencias de viajeros

Propuesta de Valor

- Generación automática inteligente con datos reales de 5+ APIs
- Comunidad integrada: publicar, likes, comentarios
- UX en 4 pasos + moderación y seguridad



Público Objetivo

Perfil Principal

Usuarios Android 9+, españoles/europeos con origen en Madrid, nivel digital básico-medio, interesados en viajes asequibles y experiencias auténticas.

Segmentos

- **Estudiantes** (16-25): mochileros con presupuesto limitado
- **Parejas jóvenes** (25-35): planes alternativos y económicos
- **Familias** (30-50): presupuestar vacaciones familiares
- **Viajeros ocasionales**: 1-2 viajes al año

Necesidades que Cubre

Claridad financiera, simplificación, inspiración, confianza en datos reales y comunidad de viajeros similares.

3. OBJETIVOS

Objetivos del Proyecto

App Android para presupuestar y planificar viajes a capitales europeas desde Madrid de forma rápida, precisa y colaborativa, integrando datos reales y comunidad social.



Autenticación Segura

Registro/login email, verificación
+13, recuperación de contraseña



Generación de Propuestas

APIs reales (vuelos, hoteles,
actividades), itinerario diario,
fallback estimado, <10s



Gestión de Viajes

Borradores, edición, publicación,
eliminación



Comunidad Social

Feed, likes, comentarios,
favoritos, reportes, bloqueo



Panel Admin

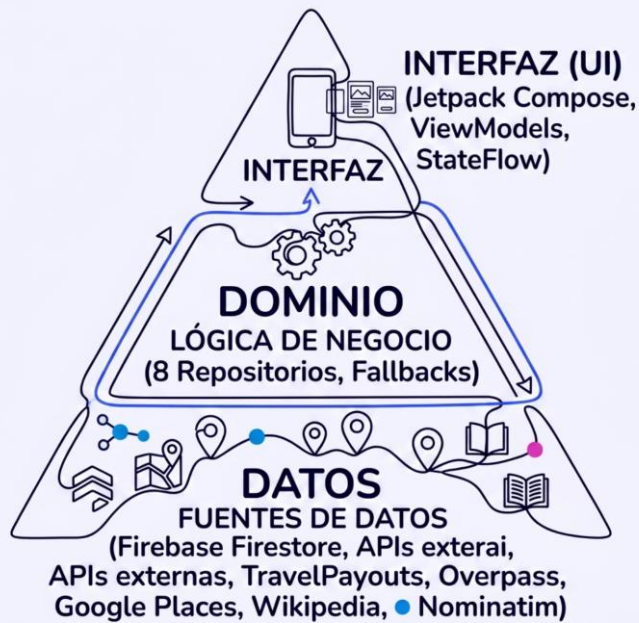
Moderar reportes, eliminar
contenido, bloquear usuarios



No Funcionales

Rendimiento ≤10s, fallback,
accesibilidad, i18n ES/EN,
Android 9+, seguridad Firebase

Tecnologías y Arquitectura



Herramientas Tecnológicas

- **Kotlin 2.2.10** + Jetpack Compose (Material 3)
- **Firebase:** Auth + Cloud Firestore
- **Arquitectura:** MVVM + Clean Architecture
- **APIs:** TravelPayouts, Overpass/OSM, Google Places, Wikipedia, Nominatim
- **Librerías:** Retrofit, Moshi, Coil, OkHttp

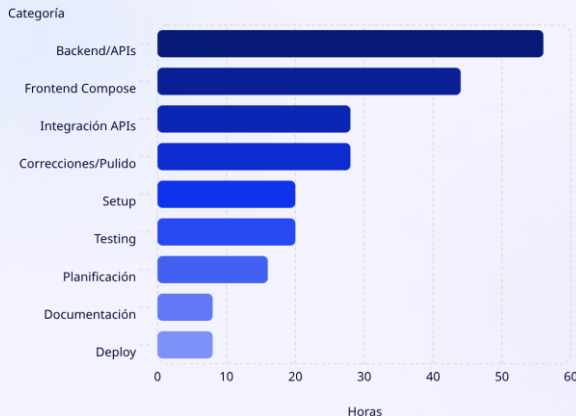
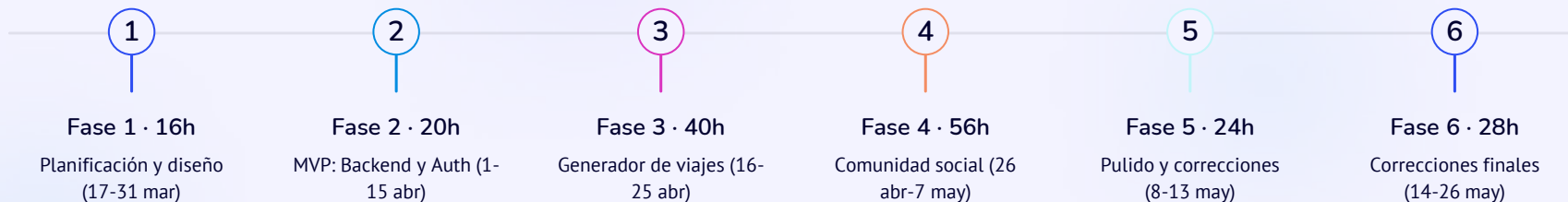
Seguridad

Firebase Auth, reglas Firestore granulares, verificación +13, HTTPS obligatorio, sistema de reportes y bloqueo de usuarios.

5. TEMPORALIZACIÓN

Plan de Desarrollo

17 mar – 27 may 2026 · 72 días · 228 horas totales (4h/día)



Distribución por Categoría

Backend y APIs lideran con 28% (56h), seguidos de Frontend con 22% (44h). Total: **228 horas**.

Hitos Clave

- **31 mar:** MVP inicial (auth, BD, diseño)
- **15 abr:** 25% funcional (formulario + APIs)
- **7 may:** 75% funcional (generador + comunidad)
- **13 may:** 100% + documentación
- **11-12 jun:** Defensa oral

Recursos del Proyecto

Humanos

Desarrolladora: Paula Izurategui · **Tutora:** Macarena Cuenca Carbajo · **Tribunal:** Tutor + 2 docentes

Espaciales

Habitación personal con escritorio, WiFi y conexión estable.

Hardware

- PC: Intel i7, 16GB RAM, SSD 500GB+
- Emulador Android 9-15 + dispositivo real
- Internet ≥ 10 Mbps

Software (todo gratuito/open source)

- Android Studio, Kotlin, Jetpack Compose
- Firebase (Auth + Firestore, tier gratuito)
- APIs: TravelPayouts, Overpass, Google Places, Wikipedia, Nominatim
- Git/GitHub, Gradle

Licencias y Cumplimiento

Código propio; librerías Apache 2.0/MIT. APIs públicas con cumplimiento de términos. **RGPD compliant**, HTTPS obligatorio.

Conclusiones y Aprendizajes

Logros Técnicos

Arquitectura MVVM + Clean Architecture escalable, fallbacks en 5+ APIs, Firestore real-time, Compose + Material 3. Desafíos superados: timeouts, concurrencia con Coroutines, performance de UI.

Requisitos Cumplidos

RF-01 a RF-31 y RNF-01 a RNF-07 implementados. Bonus: dark mode, i18n inglés, bloqueo de usuarios, edición de viajes, favoritos en perfil.

Aprendizajes Clave

Arquitectura temprana facilita mantenimiento; APIs externas son impredecibles (siempre fallback); Kotlin simplifica el código; Firebase acelera desarrollo; transparencia en fallos genera confianza.

Futuro

Corto: reserva directa, notificaciones push, histórico de viajes. **Mediano:** iOS/web, 50+ destinos, guías offline. **Largo:** ML personalizado, partnerships, gamificación.

Referencias del Proyecto

Documentación Oficial

- Android Developers: Jetpack Compose, Architecture Components
- Kotlin Language Reference
- Firebase: Firestore, Auth, Security Rules

Librerías

- Retrofit, OkHttp, Moshi (Square)
- Coil (imagen loading)

Arquitectura

- Martin, R.C. *Clean Architecture*
- Fowler, M. *Patterns of Enterprise Application Architecture*

APIs Externas

- TravelPayouts, Overpass/OSM, Google Places
- Wikipedia API, Nominatim (OSM Geocoding)

Recursos Educativos

- Android Developers Training, Jetpack Compose Pathway
- Udacity: Kotlin Bootcamp

Repositorio

github.com/Paulaizurrategui/ProyectoFinDeGrado



IA: GitHub Copilot usado como apoyo; todo el código fue revisado, comprendido y validado por la alumna.



URTRIPLY

PROYECTO FINAL

¡Gracias por su Atención!

Ha sido un placer presentarles el proyecto **UrTriply** y compartir nuestra visión para el futuro de la planificación de viajes.

Planifica, descubre y vive mejores viajes.

Explora

Reserva

Descubre

Viaja